Design Patterns in SDU Bağdar

Strategy · Adapter · Decorator — why each one, and where it lives in the Java code

1. Strategy — office-hour auto-booking (CampusPlus)

> **Why this pattern.** A student asks the system to pick an office-hour slot, but "best slot" has several valid meanings: the first one in the week, the soonest one before a deadline, the one closest to a preferred time. Without a pattern this becomes an `if/else` chain inside `OfficeHourService` that must be edited for every new rule. Strategy puts each policy behind one interface and lets the caller choose it at runtime by name (`"strategy": "deadline"` in the request body). Adding a policy means one new class and one `register()` call; existing code is untouched (Open/Closed).

Interface (the contract)

`services/campus-plus-java/src/main/java/campusos/campusplus/domain/SchedulingStrategy.java:6-11`

```
/** Strategy: one policy for picking an office-hour slot among those with free seats. */
public interface SchedulingStrategy {
    String name();

    Optional<OfficeHour> choose(List<OfficeHour> openSlots, SchedulingRequest request);
}
```

Concrete strategies

`services/campus-plus-java/src/main/java/campusos/campusplus/domain/FirstAvailableStrategy.java:8-15`

```
public final class FirstAvailableStrategy implements SchedulingStrategy {
    @Override public String name() { return "first"; }

    @Override
    public Optional<OfficeHour> choose(List<OfficeHour> openSlots, SchedulingRequest request) {
        return
openSlots.stream().min(Comparator.comparing(OfficeHour::day).thenComparing(OfficeHour::start));
    }
}
```

`services/campus-plus-java/src/main/java/campusos/campusplus/domain/EarliestDeadlineStrategy.java:8-17`

```
public final class EarliestDeadlineStrategy implements SchedulingStrategy {
    @Override public String name() { return "deadline"; }

    @Override
    public Optional<OfficeHour> choose(List<OfficeHour> openSlots, SchedulingRequest request) {
        return openSlots.stream()
                .filter(slot -> request.deadline() == null ||
!slot.nextOccurrence(request.today()).isAfter(request.deadline()))
                .min(Comparator.comparing((OfficeHour slot) ->
slot.nextOccurrence(request.today())).thenComparing(OfficeHour::start));
    }
}
```

Third implementation: `services/campus-plus-java/src/main/java/campusos/campusplus/domain/PreferredTimeStrategy.java`

Runtime selection

services/campus-plus-java/src/main/java/campusos/campusplus/application/StrategyRegistry.java:10-25

```
public final class StrategyRegistry {
    private final Map<String, SchedulingStrategy> strategies = new LinkedHashMap<>();

    public StrategyRegistry register(SchedulingStrategy strategy) {
        strategies.put(strategy.name(), strategy);
        return this;
    }

    public SchedulingStrategy get(String name) {
        SchedulingStrategy strategy = strategies.get(name);
        if (strategy == null) throw ApiException.invalid("Unknown scheduling strategy '" + name +
"'. Use one of " + names() + ".");
        return strategy;
    }

    public Set<String> names() { return strategies.keySet(); }
}
```

Context that uses it

services/campus-plus-java/src/main/java/campusos/campusplus/application/OfficeHourService.java:42-52

```
    /** Picks a slot of {@code teacherId} with the chosen strategy, then books it. */
    public OfficeHour autoBook(AuthUser student, String teacherId, String strategyName,
SchedulingRequest request) {
        SchedulingStrategy strategy = strategies.get(strategyName);
        List<OfficeHour> open = slots.listByTeacher(teacherId, student.id()).stream()
                .filter(OfficeHour::hasFreeSeat)
                .filter(slot -> !slot.bookedByMe())
                .toList();
        OfficeHour chosen = strategy.choose(open, request)
                .orElseThrow(() -> ApiException.conflict("No office hour matches the '" +
strategyName + "' policy."));
        return book(student, chosen.id());
    }
```

Wiring

services/campus-plus-java/src/main/java/campusos/campusplus/presentation/CampusPlusMain.java:42-45

```
        StrategyRegistry strategies = new StrategyRegistry()
                .register(new FirstAvailableStrategy())
                .register(new EarliestDeadlineStrategy())
                .register(new PreferredTimeStrategy());
```

Extra instance: ProjectForge (Python) uses the same pattern for recommendations (`RecommendationStrategy`).

2. Adapter — legacy university timetable (CampusPlus)

> **Why this pattern.** Room availability must come from the university's existing timetable system, which we cannot and should not change. Its interface is incompatible with ours: pipe-separated text rows, numeric weekdays and `HHMM` integers, whereas the application wants `FreeInterval` objects per room and day. Adapter translates between the two so the application depends only on the `TimetableProvider` port. If the timetable source changes (or is replaced by a real API), only the adapter is rewritten.

Target interface (what the application expects)

`services/campus-plus-java/src/main/java/campusos/campusplus/domain/TimetableProvider.java:6-9`

```
/** Port the application depends on; how the university timetable is fetched is an infrastructure
detail. */
public interface TimetableProvider {
    List<FreeInterval> freeIntervals(String roomId, DayOfWeek day);
}
```

Adaptee — legacy system, deliberately left untouched

`services/campus-plus-java/src/main/java/campusos/campusplus/infrastructure/legacy/LegacyTimetableSystem.java:12-24`

```
public final class LegacyTimetableSystem {
    private static final int[][] SLOTS = {{900, 950}, {1000, 1050}, {1100, 1150}, {1200, 1250},
            {1300, 1350}, {1400, 1450}, {1500, 1550}, {1600, 1650}};
    private final List<String> roomCodes;

    public LegacyTimetableSystem(List<String> roomCodes) { this.roomCodes = roomCodes; }

    /** Row format: ROOM|WEEKDAY|HHMM_FROM|HHMM_TO|COURSE */
    public List<String> exportScheduleRows() {
        List<String> rows = new ArrayList<>();
        roomCodes.forEach(room -> addRoomRows(rows, room));
        return rows;
    }
```

Adapter — converts rows into free intervals

`services/campus-plus-java/src/main/java/campusos/campusplus/infrastructure/legacy/LegacyTimetableAdapter.java:14-40`

```
/** Adapter: presents the legacy text export as lesson-free room intervals, leaving the legacy
class untouched. */
public final class LegacyTimetableAdapter implements TimetableProvider {
    private record Lesson(LocalTime start, LocalTime end) {}

    private final Map<String, List<Lesson>> lessonsByRoomAndDay;

    public LegacyTimetableAdapter(LegacyTimetableSystem legacy) {
        this.lessonsByRoomAndDay = legacy.exportScheduleRows().stream()
                .map(row -> row.split("\\|"))
                .collect(Collectors.groupingBy(
                        cols -> key(cols[0], DayOfWeek.of(Integer.parseInt(cols[1]))),
                        Collectors.mapping(cols -> new Lesson(parseHhmm(cols[2]),
parseHhmm(cols[3])), Collectors.toList())));
    }

    @Override
    public List<FreeInterval> freeIntervals(String roomId, DayOfWeek day) {
        List<Lesson> lessons = lessonsByRoomAndDay.getOrDefault(key(roomId, day),
List.of()).stream()
                .sorted(Comparator.comparing(Lesson::start)).toList();
        List<FreeInterval> free = new ArrayList<>();
        LocalTime cursor = RoomService.DAY_OPENS;
        for (Lesson lesson : lessons) {
            if (lesson.start().isAfter(cursor)) free.add(new FreeInterval(cursor,
lesson.start()));
            if (lesson.end().isAfter(cursor)) cursor = lesson.end();
        }
        if (cursor.isBefore(RoomService.DAY_CLOSES)) free.add(new FreeInterval(cursor,
RoomService.DAY_CLOSES));
        return free;
    }
```

Wiring

`services/campus-plus-java/src/main/java/campusos/campusplus/presentation/CampusPlusMain.java:40-40`

```
        var timetable = new LegacyTimetableAdapter(new LegacyTimetableSystem(roomCodes));
```

Consumer: `services/campus-plus-java/src/main/java/campusos/campusplus/application/RoomService.java` receives a `TimetableProvider` and never sees the legacy format.

3. Decorator — report processing pipeline (FixMyCampus)

> **Why this pattern.** Submitting a problem report needs several independent extras around the core "save the report" step: content validation with an audit trail, duplicate detection with an alert to the original reporter, and a confirmation notification. Putting all of it into one service creates a large class where steps cannot be reordered, switched off or tested alone. With Decorator each concern is a small wrapper implementing the same `ReportProcessor` interface; they are stacked in any combination, and the order is configurable via `FIXMYCAMPUS_PIPELINE` without code changes.

Component interface

`services/fix-my-campus-java/src/main/java/campusos/fixmycampus/domain/ReportProcessor.java:4-6`

```
public interface ReportProcessor {
    Report process(Report report);
}
```

Concrete component — the undecorated core

`services/fix-my-campus-java/src/main/java/campusos/fixmycampus/application/processing/BaseReportProcessor.java:8-15`

```
public final class BaseReportProcessor implements ReportProcessor {
    private final ReportRepository reports;

    public BaseReportProcessor(ReportRepository reports) { this.reports = reports; }

    @Override
    public Report process(Report report) { return reports.insert(report); }
}
```

Decorators

`services/fix-my-campus-java/src/main/java/campusos/fixmycampus/application/processing/ValidationAuditDecorator.java:24-34`

```
    @Override
    public Report process(Report report) {
        String problem = findProblem(report);
        if (problem != null) {
            audit.record(null, report.reporterId(), "REJECTED", problem);
            throw ApiException.invalid(problem);
        }
        Report saved = next.process(report);
        audit.record(saved.id(), saved.reporterId(), "SUBMITTED", saved.category() + " in " +
saved.roomId());
        return saved;
    }
```

`services/fix-my-campus-java/src/main/java/campusos/fixmycampus/application/processing/DuplicateAlertDecorator.java:20-28`

```
    @Override
    public Report process(Report report) {
        var duplicate = reports.findOpenDuplicate(report.roomId(), report.category());
        Report saved = next.process(duplicate.map(original ->
report.withDuplicateOf(original.id())).orElse(report));
        duplicate.filter(original -> !original.reporterId().equals(report.reporterId()))
                .ifPresent(original -> notifications.add(original.reporterId(),
                        "Someone else reported the same " + original.category() + " problem in " +
original.roomId() + "."));
        return saved;
    }
```

services/fix-my-campus-java/src/main/java/campusos/fixmycampus/application/processing/NotificationDecorator.java:17-22

```
    @Override
    public Report process(Report report) {
        Report saved = next.process(report);
        notifications.add(saved.reporterId(), "Report " + saved.id() + " received: " +
saved.category() + " in " + saved.roomId() + ".");
        return saved;
    }
```

Composition: stacking decorators

services/fix-my-campus-java/src/main/java/campusos/fixmycampus/application/processing/ReportPipelineFactory.java:24-37

```
    public ReportProcessor build(List<String> names) {
        ReportProcessor processor = new BaseReportProcessor(reports);
        for (String name : names.reversed()) processor = wrap(name.trim(), processor);
        return processor;
    }

    private ReportProcessor wrap(String name, ReportProcessor inner) {
        return switch (name) {
            case "validation" -> new ValidationAuditDecorator(inner, categories, audit);
            case "duplicate" -> new DuplicateAlertDecorator(inner, reports, notifications);
            case "notification" -> new NotificationDecorator(inner, notifications);
            default -> throw new IllegalArgumentException("Unknown report decorator '" + name +
"'");
        };
    }
```

Wiring

services/fix-my-campus-java/src/main/java/campusos/fixmycampus/presentation/FixMyCampusMain.java:31-32

```
        var pipeline = new ReportPipelineFactory(reports, support, support, support)
                .build(Arrays.asList(Env.get("FIXMYCAMPUS_PIPELINE",
"validation,duplicate,notification").split(",")));
```

With the default `validation,duplicate,notification` a report is validated first, then checked for duplicates, then saved by the base processor; notifications are sent on the way back out.

Summary

Pattern	Problem solved	Key files
Strategy	Interchangeable slot-picking algorithms	`SchedulingStrategy`, `StrategyRegistry`, `*Strategy`
Adapter	Incompatible legacy timetable format	`TimetableProvider`, `LegacyTimetableAdapter`
Decorator	Stackable report-handling steps	`ReportProcessor`, `*Decorator`, `ReportPipelineFactory`

Tests: `tests/run-java-tests.sh` covers all three patterns.